

سوال ۱ - آشوب در کارخانه ویلی وانکا



در کارخانه شکلات‌سازی ویلی وانکا، اومپا-لومپاها (اون کارگرهای کوچولو و بامزه) مسئول مرتب کردن جعبه‌های شکلات بر اساس وزن هستند. اما ناگهان، ویلی وانکا متوجه می‌شه که خط تولید بهم ریخته و جعبه‌ها نامرتب شدن! ویلی فریاد می‌زنه: اومپاها! باید سریع‌تر از همیشه مرتب کنید، چون چارلی و پدر بزرگش دارن میان بازدید و نباید آشوب ببینن!

اومپاها تصمیم می‌گیرن از یک الگوریتم مرتب‌سازی حبابی پیشرفته استفاده کنن: مرتب‌سازی کوکتل شیکر (Cocktail Shaker Sort). این روش مثل یک کوکتل میکسر کار می‌کنه: از دو طرف آرایه رو مرتب می‌کنه تا حباب‌های بزرگ و کوچک سریع‌تر جابجا بشن. (در واقع مثل یه شیکر واقعی، عناصر در این الگوریتم هم اول به یک سمت بالا می‌رن و بعد دوباره برمی‌گردن پایین! 🍹💎)

- وظیفه شما: الگوریتم Cocktail Shaker Sort رو پیاده کنید تا لیست وزن جعبه‌های شکلات رو مرتب کنه (صعودی). علاوه بر مرتب کردن، تعداد مقایسه‌ها

(هر بار که دو عنصر رو مقایسه می‌کنید) و تعداد جابجایی‌ها (هر بار که swap انجام می‌شه) رو هم بشمارید.

این الگوریتم یک واریاسیون از *bubble sort* هست که در هر پاس، یک بار از چپ به راست و یک بار از راست به چپ جابجایی انجام می‌ده تا عناصر سریع‌تر به جایگاه درست برسند.

راهنمایی:

- مثل *bubble sort* معمولی، اما دوجهته: اول از ابتدای آرایه به انتها swap کنید (برای حباب‌های بزرگ)، بعد از انتها به ابتدا (برای حباب‌های کوچک).
 - یک فلگ برای چک کردن اینکه آیا swap انجام شده یا نه، اضافه کنید تا اگر آرایه زودتر مرتب شد، حلقه متوقف بشه.
 - برای شمارش: متغیرهایی مثل $comparisons = 0$ و $swaps = 0$ تعریف کنید و هر بار مقایسه یا swap، یکی اضافه کنید.
 - پیچیدگی زمانی: تحلیل کنید که در بهترین حالت (آرایه مرتب از قبل) $O(n)$ مقایسه داره، در بدترین حالت (معکوس مرتب) $O(n^2)$ مقایسه و swap، و در متوسط $O(n^2)$. این تحلیل رو می‌تونید در کامنت‌های کد بنویسید، اما خروجی فقط اعداد باشه.
- ورودی: آرایه ای نامرتب از اعداد (وزن جعبه ها)

خروجی :

- خط اول: آرایه مرتب شده با فرمت لیست پایتون (مثل [1,2,3,4,5,6,7,8])
- خط دوم: تعداد مقایسه‌ها (یک عدد صحیح)
- خط سوم: تعداد جابجایی‌ها (یک عدد صحیح)

ورودی نمونه:

5 2 6 4 7 1 3 8

خروجی مورد انتظار:

[1, 2, 3, 4, 5, 6, 7, 8]

21

10

(البته! در این ورودی، تعداد مقایسه‌ها و جابجایی‌ها بسته به پیاده‌سازی دقیق می‌تونه کمی متفاوت باشه، اما برای این نمونه فرض کنیم ۲۱ مقایسه و ۱۰ swap انجام شده.)

سوال ۲ - نفوذ به سیستم امنیتی ماتریکس



در دنیای ماتریکس، نئو متوجه می‌شود که Agent ها از طریق درگاه‌های امنیتی مختلف وارد سیستم می‌شن. برای شناسایی الگوی حمله و پیش‌بینی نقطه نفوذ بعدی، نئو باید **زمان‌های ورود** ایجنت‌ها (به ثانیه) رو تحلیل کنه.

مشکل اینجاست: این زمان‌ها به شدت بهم‌ریخته ثبت شده‌ن و Agent ها دارن نزدیک می‌شن! نئو باید سریع این داده‌ها رو مرتب کنه.

مورفیوس می‌گه: **نئو! از shell sort استفاده کن. این الگوریتم مثل Insertion Sort کار می‌کنه، ولی با یک هک هوشمندانه که سرعتش رو چند برابر می‌کنه!**

مرتب‌سازی شل (Shell Sort) چیه؟

مشکل Insertion Sort معمولی

Insertion Sort یک الگوریتم ساده برای مرتب‌سازی. نحوه کارش:

- از اول آرایه شروع می‌کنه
- هر عنصر رو برمی‌داره و در جای مناسبش در قسمت مرتب‌شده قرار می‌ده
- برای این کار، عنصر رو با عناصر قبلی یکی یکی مقایسه می‌کنه

مثال:

آرایه: [50, 60, 70, 80, 10]

مرحله 1: [50] | 60, 70, 80, 10

مرحله 2: [50, 60] | 70, 80, 10

مرحله 3: [50, 60, 70] | 80, 10

مرحله 4: [50, 60, 70, 80] | 10

مرحله 5: حالا باید 10 رو جا بدیم

- $10 < 80 \rightarrow \text{swap} \rightarrow [50, 60, 70, 10, 80]$

- $10 < 70 \rightarrow \text{swap} \rightarrow [50, 60, 10, 70, 80]$

- $10 < 60 \rightarrow \text{swap} \rightarrow [50, 10, 60, 70, 80]$

- $10 < 50 \rightarrow \text{swap} \rightarrow [10, 50, 60, 70, 80]$

مشکل: عدد 10 باید 4 بار جابجا بشه تا به ابتدا برسه. اگر آرایه بزرگ باشه و عناصر کوچک در انتها باشن، خیلی کند می‌شه!

Shell Sort : راه حل

دونالد شل (Donald Shell) در سال 1959 فکری کرد: چرا همیشه فقط یک قدم جلو بریم؟ چرا با قدم‌های بزرگ شروع نکنیم؟

نحوه کار Shell Sort

مرحله 1 - انتخاب فاصله اولیه (gap): معمولا از $n/2$ شروع می‌کنیم (n تعداد عناصر)

مرحله 2 = Insertion Sort: با فاصله gap عناصری که به اندازه gap از هم فاصله دارند رو مثل یک زیرآرایه مرتب می‌کنیم.

مرحله 3 - کاهش gap: gap رو نصف می‌کنیم و مرحله 2 رو تکرار می‌کنیم.

مرحله 4 - ادامه تا $gap = 1$: وقتی gap برابر 1 شد، دقیقا مثل Insertion Sort معمولی می‌شه، ولی چون آرایه تقریبا مرتب شده، خیلی سریع‌تر کار می‌کنه!

مثال کامل

آرایه

[35, 33, 42, 10, 14, 19, 27, 44]

گام 1 : $gap = 8/2 = 4$

زیرآرایه‌ها:

- [35, 14] (ایندکس 0, 4)
- [33, 19] (ایندکس 1, 5)
- [42, 27] (ایندکس 2, 6)
- [10, 44] (ایندکس 3, 7)

بعد از مرتب‌سازی هر زیرآرایه:

[14, 19, 27, 10, 35, 33, 42, 44]

گام 2 : $gap = 4/2 = 2$

زیرآرایه‌ها:

- [14, 27, 35, 42] (ایندکس 0, 2, 4, 6)
- [19, 10, 33, 44] (ایندکس 1, 3, 5, 7)

بعد از مرتب‌سازی:

[14, 10, 27, 19, 35, 33, 42, 44]

گام 3 : $gap = 2/2 = 1$

معمولی Insertion Sort حالا

[10, 14, 19, 27, 33, 35, 42, 44]

وظیفه شما

الگوریتم Shell Sort را پیاده‌سازی کنید که:

1. آرایه زمان‌های ورود را مرتب کند (صعودی)
2. تعداد مقایسه‌ها را بشمارد (هر بار که دو عنصر مقایسه می‌شوند)
3. تعداد جابجایی‌ها را بشمارد (هر بار که swap انجام می‌شود)

ورودی

آرایه ای نامرتب از زمان های ثبت شده

خروجی

- خط اول: آرایه مرتب شده (فرمت لیست پایتون)
- خط دوم: تعداد مقایسه ها (int)
- خط سوم: تعداد جابجایی ها (int)

نمونه ورودی:

29 10 14 37 13 25 18

نمونه خروجی:

[10, 13, 14, 18, 25, 29, 37]

16

8

سوال ۳ - پیچیدگی زمانی کد و چطوری حسابش کنیم؟

دوباره آخر ترم شده و کابوس مشروطی مثل یه سایه سیاه بر سر بعضی از دانشجویها سنگینی می‌کرد. کیهان که از بچه‌های رشته کامپیوتر بود، این بار واقعا در موقعیت بحرانی قرار داشت! "اگه این ترم هم مشروط بشم، اخراج می‌شم. باید هر کاری بکنم تا امتحانا رو پاس کنم!"

🔑 خبر مهم: کلید گاوصندوق!

دقیقا در همین موقع، شایعه‌ای در دانشکده پیچید: "یکی از بچه‌ها کلید گاوصندوق سوالات امتحان رو گیر آورده 🦉!"

کیهان چشم‌هاش برق زد "اگه بتونم پیداش کنم، می‌تونم سوالا رو ببینم و ... کیهان نگاهی به کلاس انداخت. 100 تا دانشجوی همکلاسی اونجا نشسته بودن و یکی از اون‌ها کلید رو داشت، ولی کیهان نمی‌دونست کی! "چطور می‌تونم سریع پیداش کنم؟" کیهان با خودش فکر کرد.

🇮🇷 روش اول $O(n^2)$: استراتژی پارانوئیدی!

میرم پیش علی (نفر اول کلاس) و می‌پرسم: 'کلید رو داری؟' بعد از همون علی درباره همه 99 نفر دیگه هم می‌پرسم: فکر می‌کنی محمد کلید رو داره؟ پس سارا چی؟ مهدی چطور؟ و همین‌طور تا آخر...

بعدش برم پیش محمد (نفر دوم) و همین کارو تکرار کنم! از محمد هم درباره همه 99 نفر بپرسم!

این کارو برای همه 100 نفر انجام بدم!

"ولی این یعنی $100 \times 100 = 10,000$ تا سوال بپرسم؟! دیوونه‌ام مگه؟"

📈 روش دوم $O(n)$: استراتژی معقول

"برم یکی یکی از همه 100 نفر بپرسم: کلید رو داری؟ نفر اول... نفر دوم... نفر سوم... تا اینکه به کسی که کلید رو داره برسم" در بدترین حالت، 100 تا سوال می‌پرسم. در حالت متوسط حدود 50 تا".

کیهان فکری کرد: "این بهتره، ولی هنوز کنده!"

🚀 روش سوم $O(\log n)$: استراتژی نابغانه!

شاید همه بچه‌ها می‌دونن کلید پیش کیه، ولی فقط وقتی سمت درست رو حدس بزنم بهم می‌گن. کلاس رو نصف میکنم، به 50 نفر سمت چپ نگاه میکنم و میپرسم: 'کلید اینجا ست؟' اگه گفتن 'آره'، پس 50 نفر سمت راست رو فراموش بیخیال میشم! اگه گفتن 'نه'، پس 50 نفر سمت چپ رو ول میکنم. حالا 50 نفر باقی مونده. دوباره نصفشون میکنم! 25 نفر... بعدش 12 نفر... بعدش 6 نفر... 3 نفر... 1 نفر! اینجا جوی فقط حدود 7 تا سوال می‌پرسم و پیدااش می‌کنم!"

پس اگه

- فقط یک نفر بدونه کلید پیش کیه $\rightarrow O(n^2)$
- کلید دار خودش بدونه ولی بقیه ندونن $\rightarrow O(n)$
- همه بدونن ولی فقط با حدس درست جواب بدن $\rightarrow O(\log n)$

Big O چیه؟

اون O که اینجا گفتیم، در واقع نماد بزرگی رشد (Big-O notation) هست. این نماد برای نمایش اینه که زمان اجرای الگوریتم چطور نسبت به اندازه ورودی بزرگ میشه. البته نمادهای دیگه‌ای مثل Θ و Ω هم وجود دارن.

ما به نرخ رشد تعداد اجرای دستورات کد نسبت به ورودی‌ها علاقه‌مندیم، نه زمان واقعی اجرا.

حالا یه سوال!

آیا پیچیدگی زمانی الگوریتم/کد برابر با زمان اجرا/پردازش کد؟

پیچیدگی زمانی یه الگوریتم/کد برابر نیست با زمان واقعی که برای اجرای یه کد خاص لازمه، بلکه برابر با تعداد دفعاتی که دستورات کد اجرا میشن. مثلاً یک خط کد ممکنه سریع یا کند اجرا بشه، ولی پیچیدگی فقط می‌گه اون خط چند بار اجرا شده.

پس به جای اینکه بگیم هر دستور چقدر زمان می‌بره، پیچیدگی زمانی می‌گه هر دستور چند بار اجرا میشه و بر اساس اون، سرعت کلی الگوریتم رو تخمین می‌زنه.

هر چی حلقه‌ها بیشتر باشن، شرط‌ها و ساختارهای در هم بیشتر باشه، پیچیدگی زمانی بیشتر میشه. اما چطوری دقیق‌تر حساب کنیم؟ باید نگاه کنیم هر خط کد چند بار اجرا میشه، جمع کنیم ببینیم کل کد چند بار عملیات انجام میده. چندتا مثال ببینیم :

مثال: تابع ساده با یک حلقه

```
def example1(array):  
    for i in array:  
        print(i)
```

خط `for i in array` : یعنی حلقه‌ای که به اندازه n بار اجرا میشه (n طول آرایه هست)

خط `print(i)` به ازای هر باری که حلقه اجرا بشه، یک بار اجرا میشه
پس کل عملیات تقریباً n بار اجرا میشه، یعنی پیچیدگی زمانی میشه $O(n)$

مثال: تابع با حلقه و شرط

```
def example2(array):  
    for i in array:  
        if i % 2 == 0:  
            print(i)
```

حلقه به اندازه n بار اجرا میشه

شرط `if` برای هر بار اجرا چک میشه، یعنی n بار

`Print` فقط وقتی اجرا میشه که شرط درست باشه، ولی برای محاسبه پیچیدگی زمانی معمولاً می‌گیم بدترین حالت، یعنی فرض می‌کنیم همیشه شرط درست باشه
پس باز هم در بدترین حالت کل عملیات تقریباً n بار اجرا میشه
پیچیدگی زمانی $O(n)$

مثال: تابع با حلقه تو در تو

```
def example3(array):  
    for i in array:  
        for j in array:  
            print(i, j)
```

حلقه بیرونی n بار اجرا میشه

برای هر بار اجرای حلقه بیرونی، حلقه درونی هم n بار اجرا میشه

پس کل `print` به تعداد $n * n = n^2$ بار اجرا میشه
پیچیدگی زمانی $O(n^2)$

مثال: تابع با حلقه و شرط داخل حلقه درونی

```
def example4(array):  
    for i in array:  
        for j in array:  
            if i == j:  
                print(i, j)
```

حلقه بیرونی n بار اجرا میشه

حلقه درونی به ازای هر i ، n بار اجرا میشه (تا اینجا $n * n$ بار یعنی n^2)

شرط `if` برای هر تکرار بررسی میشه، یعنی n^2 بار

خط `print` در بدترین حالت فرض می‌کنیم اجرا میشه (مثلا همه مقایسه‌ها درست) پس n^2 بار
پیچیدگی زمانی $O(n^2)$

پس:

1. **حلقه‌ها:** هر حلقه که به اندازه n اجرا میشه، پیچیدگی رو حداقل ضربدر n می‌کنه.

2. **دستورات ساده:** خطوطی مثل انتساب، چاپ، شرط ساده که داخل حلقه نیستن، معمولا $O(1)$ هستن (ثابت) چون فقط یه بار اجرا میشن.

3. **شرط‌ها:** اگر شرط داخل حلقه باشه، به اندازه تعداد تکرار حلقه اجرا میشه.

4. **حلقه‌های تودرتو**: به تعداد کل ترکیبات حلقه‌ها ضربدر n میشن. مثلاً دو حلقه تو در تو همیشه $O(n^2)$ ، سه حلقه $O(n^3)$ و الی آخر.

حالا نوبت شماست!
پیچیدگی زمانی قطعه کدهای زیر رو در بدترین حالت محاسبه کنید و استدلالتون رو بطور کوتاه توضیح بدید.

1)

```
def fun(N):  
    counter=1  
    for i in range(1,N+1):  
        for j in range(1,i+1):  
            counter+=1  
    print(counter)
```

2)

```
def fun(N):  
    for i in range(1,N+1):  
        j=N  
        while j>0:  
            j//=2
```

3)

```
a = 0
b = 0
for i in range(N):
    a = a + random()

for i in range(M):
    b = b + random()
```

4)

```
function conundrum(n)
    r:=0;
    for i:= 1 to n do
        for j:= i + 1 to n do
            for k:= i+j-1 to n do
                r:= r+1
    return(r)
```

5)

```
def example4(N):
    counter = 0
    i = 1
    while i < N:
        for j in range(i):
            counter += 1
        i *= 3
    return counter
```

سول ۴ - پیچیدگی مجانبی (Asymptotic Complexity)

ما معمولا وقتی می‌خواهیم بدونیم به الگوریتم چقدر طول می‌کشد (یا چقدر حافظه مصرف می‌کند) وقتی ورودی خیلی بزرگ می‌شود، می‌ریم سراغ «نمادهای مجانبی». سه تا از مهم‌ترین‌هاش:

Big O

اگر بگیم $g(n) \in O(f(n))$ ، یعنی وقتی n خیلی بزرگ شود، تابع g از f کندتر رشد می‌کند یا حداکثر به اندازه f رشد می‌کند. به زبان ساده: یعنی g کوچک‌تر یا مساوی f رشد می‌کند.

مثال: $n \in O(n^2)$

Big-Omega (Ω)

اینجا برعکس! $g(n) \in \Omega(f(n))$ ، یعنی تابع g از f سریع‌تر رشد می‌کند یا حداقل به اندازه f رشد می‌کند. به زبان ساده: یعنی g بزرگ‌تر یا مساوی f رشد می‌کند.

مثال: $n^2 \in \Omega(n)$

Big-Theta (Θ)

اگر $g(n) \in \Theta(f(n))$ ، یعنی رشد g تقریبا برابر با رشد f هست — نه بیشتر، نه کمتر. به زبان ساده: یعنی g و f سرعت رشدشون تقریبا مساویه و تو یه بازه مشخص، این دو تا با هم دیگه scale می‌شن.

مثال: $5n \in \Theta(n)$

حالا نوبت شماست! به سوالات زیر پاسخ دهید.

- تابع‌های زیر رو بر اساس کلاس Big-O ، از کندترین رشد تا سریع‌ترین مرتب کنید.

$$100n + \log n, \log_{100} n, 3n^2, (n!)^2, (n^2)!, 1/n$$

دو تابع $f, g : \mathbb{N} \rightarrow \mathbb{N}$ بیابید، که اکیدا صعودی باشند و داشته باشیم:

$$g(n) \notin O(f(n)), f(n) \notin O(g(n))$$

- $\Theta(n!)$ را محاسبه کنید.

- درستی یا نادرستی عبارت زیر را بررسی کنید.

اگر $f(n) \in O(g(n))$ و $f(n) \in \tilde{O}(g(n))$ آنگاه $f(n) \in \Theta(g(n))$